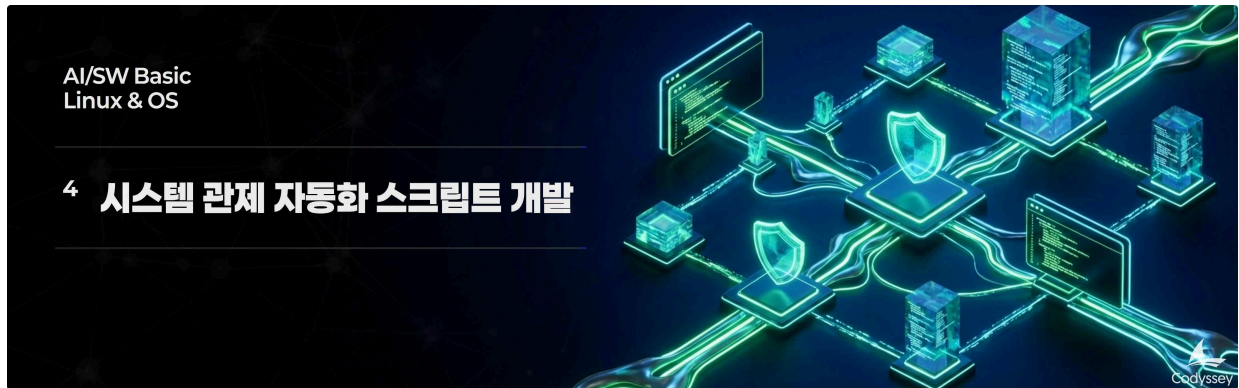


분야 AI/SW 기초 구분 Linux와 OS 학습시간 40시간

시스템 관제 자동화 스크립트 개발

문제기술

기술적 설명



1. 미션 소개

서버 장애가 났을 때 로그가 없으면, 원인 분석은 '감'에 의존하게 됩니다. 실제 현업에서 이런 상황이 발생하면 복구 시간이 수 배로 늘어나는 건 물론이고, 같은 장애가 반복됩니다. 권한 관리, 네트워크 보안, 로그 자동화까지 서버를 운영하는 엔지니어처럼 직접 설계합니다.

리눅스는 현대 서버 개발 및 운영 환경의 표준 운영체제 중 하나입니다. 단순히 명령어를 암기하는 1회성 학습이 아니라, 개발 커리어 내내 활용 가능한 안정적인 서버 운영 환경을 직접 구축해 보는 것이 핵심입니다.

이 미션에서는 다중 사용자 환경에서의 권한 관리와 네트워크 보안 설정을 시작으로, 실제 서비스를 배포하고 운영할 때 필수적인 시스템 리소스 관제와 로그 관리를 자동화하는 셸 스크립트 개발을 수행합니다.

최종적으로 단순한 리눅스 사용자를 넘어, 애플리케이션 배포 환경을 구축하고 시스템의 상태를 관제하며 데이터로 기록할 수 있는 엔지니어링 역량을 갖추게 됩니다.

2. 최종 결과물

다음 2가지 산출물을 제출해야 한다.

1. 요구사항 수행 내역서(문서 1개)

- 수행 내역

- 설정/명령어 기록 (SSH 포트, 방화벽 규칙, 계정/그룹/ACL, 디렉토리/권한, 환경 변수, cron 등록 등)
- 필수 증거 자료 체크리스트
 - SSH 포트 변경(20022) 및 Root 원격 접속 차단 설정 확인 내역
 - 방화벽(UFW 또는 firewalld) 활성화 및 20022/tcp, 15034/tcp만 허용 내역
 - 계정/그룹(agent-admin/dev/test, agent-common/core) 생성 확인 내역
 - 디렉토리 구조 및 권한(ACL 포함) 확인 내역
 - 앱 Boot Sequence 5단계 [OK] 및 “Agent READY” 확인 내역
 - monitor.sh 실행 결과(프로세스/포트/리소스/경고) 내역
 - /var/log/agent-app/monitor.log 누적 기록 확인(최근 라인) 내역
 - crontab 매분 실행 등록 및 자동 실행 확인(1분 후 로그 증가) 내역

2. 자동화 스크립트 소스코드

- monitor.sh : 시스템 상태 수집 및 로깅 스크립트

3. 과제 목표

이 과제를 마친 후, 학습자는 아래를 스스로 설명할 수 있어야 한다.

- SSH 포트 변경과 Root 원격 접속 차단이 왜 기본 보안에 해당하는지 설명할 수 있다.
- UFW 또는 firewalld 중 하나를 선택해 “필요 포트만 허용”하는 방화벽 정책을 구성하고 검증할 수 있다.
- 역할 기반 계정/그룹과 ACL을 통해 “공유 디렉토리”와 “보안 디렉토리”를 분리하는 이유를 설명할 수 있다.
- 환경 변수(AGENT_HOME 등)로 실행 환경을 고정하는 이유와 검증 방법을 설명할 수 있다.
- 쉘 스크립트로 프로세스/포트/리소스 상태를 수집하고, 로그로 남겨 운영 문제를 추적하는 흐름을 설명할 수 있다.
- crontab으로 모니터링을 주기 실행시키고, 로그 보존 정책(압축/삭제)이 왜 필요한지 설명할 수 있다.

4. 기능 요구 사항

다음 요구사항을 모두 만족해야 한다.

1. 기본 보안 및 네트워크 설정

- SSH 설정
 - SSH 접속 포트를 20022로 변경한다.
 - Root 원격 로그인을 차단한다.
 - 확인 방법(예시)
 - sshd 설정 파일에서 포트/PermitRootLogin 확인
 - 포트 리슨 상태 확인: `ss -tulnp` 후 sshd 관련 라인 확인
- 방화벽 설정(택1)
 - UFW 또는 firewalld 중 하나를 선택해 활성화한다.
 - 인바운드 허용 포트는 TCP 20022(SSH), TCP 15034(APP)만 허용한다.
 - 확인 방법(예시)
 - UFW 선택 시: `ufw status`
 - firewalld 선택 시: `firewall-cmd --list-all`

2. 계정/그룹/권한 체계(협업 + 최소 권한)

- 생성 계정
 - agent-admin (운영/관리, cron 실행자)
 - agent-dev (개발/운영, monitor.sh 작성자)
 - agent-test (QA/테스트)
- 생성 그룹
 - agent-common: admin, dev, test
 - agent-core: admin, dev
- 디렉토리 구조(AGENT_HOME 기준)
 - \$AGENT_HOME
 - \$AGENT_HOME/upload_files
 - \$AGENT_HOME/api_keys
 - /var/log/agent-app
- 접근 권한(핵심 정책)
 - upload_files: group=agent-common, R/W 가능
 - api_keys 및 /var/log/agent-app: group=agent-core ONLY, R/W 가능

- 확인 방법(예시)

- id agent-admin / id agent-dev / id agent-test
- ls -l 및 getfacl(사용 시)로 소유/권한 확인

3. 애플리케이션 실행 환경 구성(제공 Python 앱)

- 환경 변수
 - AGENT_HOME: 예) /home/agent-admin/agent-app
 - AGENT_PORT: 15034
 - AGENT_UPLOAD_DIR: \$AGENT_HOME/upload_files
 - AGENT_KEY_PATH: \$AGENT_HOME/api_keys/t_secret.key
 - AGENT_LOG_DIR: /var/log/agent-app (미지정 시 기본값이므로 지정 권장)
- 키 파일 생성
 - 경로: \$AGENT_HOME/api_keys/t_secret.key
 - 내용: agent_api_key_test (1줄)
- 앱 실행 및 성공 기준
 - 일반 계정으로 실행(루트 실행 금지)
 - Boot Sequence 5단계가 모두 [OK]로 출력되고, 마지막에 “Agent READY”가 출력되어야 한다.
 - 앱이 0.0.0.0:15034로 LISTEN 상태가 되어야 한다.
 - 참고: 앱 종료는 Ctrl+C로 수행한다.

4. 시스템 관제 자동화 스크립트(monitor.sh) 구현

- 파일 위치/권한 정책
 - 경로: \$AGENT_HOME/bin/monitor.sh
 - 소유자: agent-dev
 - 그룹: agent-core
 - 권한: 750 (rwxr-x---)
 - cron 실행 계정: agent-admin (agent-admin은 agent-core에 포함되어 실행 가능해야 함)
- Health Check(실패 시 종료)

- 프로세스: agent_app.py(또는 제공 앱 파일명) 실행 상태를 확인하고, 비정상 시 exit 1
- 포트: TCP 15034 LISTEN 상태 확인, 비정상 시 exit 1
- 상태 점검(경고만 출력)
 - 방화벽(UFW 또는 firewalld) 활성화 상태를 점검한다.
 - 비활성 상태면 [WARNING]을 출력하되, 스크립트는 종료하지 않는다.
- 자원 수집
 - CPU 사용률(%)
 - 메모리 사용률(%)
 - 디스크 사용률(Root partition, Used %)
- 임계값 경고(경고만 출력)
 - CPU > 20%: [WARNING]
 - MEM > 10%: [WARNING]
 - DISK_USED > 80%: [WARNING]
- 로그 기록
 - 로그 파일: /var/log/agent-app/monitor.log
 - 로그 포맷
 - [YYYY-MM-DD HH:MM:SS] PID:... CPU:...% MEM:...%
DISK_USED:...%
- 로그 파일 용량 관리
 - monitor.log가 커지면 최대 10MB/10개 파일 유지(방법 자유: logrotate 사용 또는 스크립트 로직 구현)

5. 자동 실행(cron) 설정

- agent-admin 계정의 crontab으로 monitor.sh를 매분 실행되도록 등록한다.
- 등록 후 1~2분 내 monitor.log에 새 라인이 자동으로 누적되는 것을 확인한다.

5. 보너스 과제 (선택)

보너스 1 – report.sh로 요약 리포트 자동 생성

- monitor.log를 분석해 CPU/MEM/DISK의 평균/최대/최소와 샘플 수를 콘솔로 출력한다.
- (선택) 시작/종료 시간을 입력받아 해당 구간의 로그만 분석한다.

보너스 2 – 시간 기반 로그 보존 정책(압축/아카이브/삭제)

- 7일 경과 로그 압축
 - 대상: /var/log/agent-app/*.log 중 7일 이상 경과 파일
- 아카이브 이동
 - 경로: /var/log/monitor/agent-app/archive/
- 30일 경과 아카이브 삭제
 - 대상: /var/log/monitor/agent-app/archive/*.gz 중 30일 이상 경과 파일
- (권장) 예외 처리 포함
 - 디렉토리 미존재, 권한 부족, 대상 파일 0개 등에서 “안전하게 종료/경고”하도록 처리

개발환경

6. 개발 환경

- Ubuntu 22.04 LTS 또는 동등 리눅스 환경
- 이전 미션에서 구성한 Linux 실습 환경(컨테이너/VM)을 그대로 사용 권장

제약조건

7. 제약 사항

- 구현 언어/도구
 - 자동화 스크립트는 Bash로만 작성한다(Python 등으로 대체 금지)
 - 필요한 경우에만 sudo 사용(가능한 일반 계정으로 진행)
- 제공 애플리케이션

- 제공된 Python 앱은 “실행 대상”이며, 과제의 핵심은 관제/자동화 스크립트 구현이다.

Test Case

8. 결과 예시

아래는 정답이 아니라 참고 예시다. 실제 문구와 구성은 달라도 된다.

- 앱 Boot Sequence 출력 예시

```
> Starting Agent Boot Sequence...
[1/5] Checking User Account      [OK]
... Running as service user 'agent-admin' (uid=1001)
[2/5] Verifying Environment Variables  [OK]
... All required Envs correct
[3/5] Checking Required Files      [OK]
... Verified key file with correct key string.
[4/5] Checking Port Availability    [OK]
... Port 15034 is available.
[5/5] Verifying Log Permission      [OK]
... Log directory is writable: /var/log/agent-app
-----
All Boot Checks Passed!
Agent READY
```

CSS

- monitor.sh 콘솔 출력 예시

```
===== SYSTEM MONITOR RESULT =====

[HEALTH CHECK]
Checking process 'agent_app.py'... [OK] (PID: 48291)
Checking port 15034... [OK]

[RESOURCE MONITORING]
CPU Usage : 25.3%
MEM Usage : 5.2%
DISK Used  : 23%
```

CSS

[WARNING] CPU threshold exceeded (25.3% > 20%)

===== STATISTICS REPORT =====

[CPU]

Average : 21.4%

Maximum : 25.3% at 2026-02-25 14:00:05

Minimum : 10.2% at 2026-02-25 13:58:05

[Memory]

Average : 6.1%

Maximum : 9.8% at 2026-02-25 14:00:05

Minimum : 3.2% at 2026-02-25 13:58:05

[Samples]

Data Points: 10 samples

[INFO] Log appended: /var/log/agent-app/monitor.log

- monitor.log 누적 예시

CSS

[2026-02-25 13:58:01] PID:48291 CPU:10.2% MEM:3.2% DISK_USED:23%

[2026-02-25 13:59:01] PID:48291 CPU:18.7% MEM:5.0% DISK_USED:23%

[2026-02-25 14:00:01] PID:48291 CPU:25.3% MEM:9.8% DISK_USED:23%

- (보너스 수행 시) report.sh 콘솔 출력 예시

CSS

===== STATISTICS REPORT =====

[CPU]

Average : 21.4%

Maximum : 25.3% at 2026-02-25 14:00:05

Minimum : 10.2% at 2026-02-25 13:58:05

[Memory]

Average : 6.1%

Maximum : 9.8% at 2026-02-25 14:00:05

Minimum : 3.2% at 2026-02-25 13:58:05

[Samples]

Data Points: 10 samples